Unified Engineering Workbench

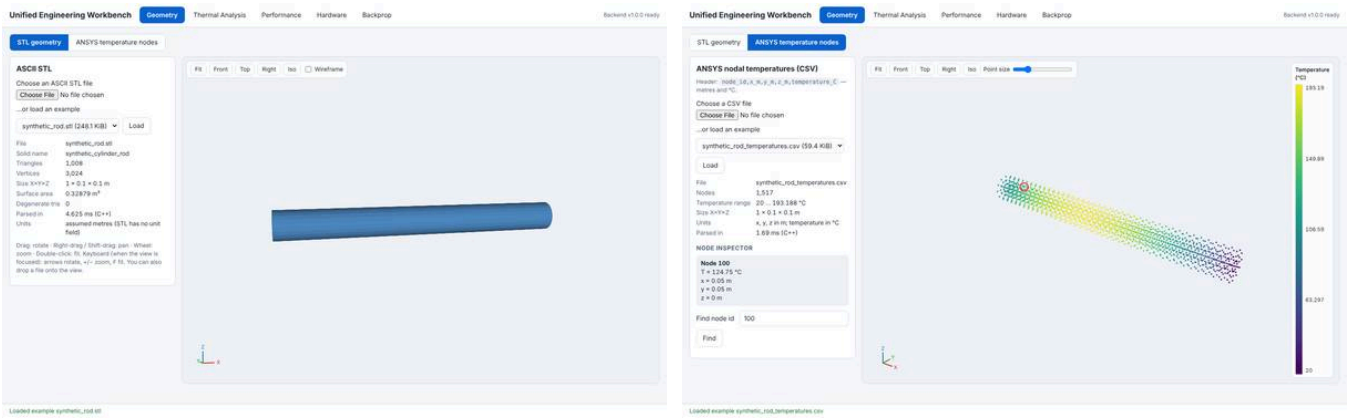
[Candidate name / ID] · PASS SCIENCE WORKS assignment · Windows 11 x64 · C++17 backend + web UI · Full package: [link to CandidateID_Workbench.zip and Git repository - add before submitting]

1. Summary and design choices

One application, five views (Geometry, Thermal, Performance, Hardware, Backprop). `Launch.exe` (tiny Win32 GUI program) starts `bin\wb_server.exe` hidden on `127.0.0.1`, waits for `/api/health` and opens an Edge app window; the backend exits itself a few seconds after the window closes. No installer, admin rights, terminal or internet.

- **Separation:** `wb_core` (C++17 library: FEA, STL/CSV parsers, thread pool, batch, device mocks, backprop) has no HTTP/UI code and is unit-tested directly; `wb_server` (cpp-httplib, MIT) only validates and serialises; the UI (plain ES modules + canvas, no CDN) only draws.
- **Why a local web UI:** works on a clean Windows 11 PC with no runtime to install; Edge is preinstalled; the same core is reused by `wb_cli`.
- **Safety:** loopback only, Host-header check, `X-WB-Client` header on every POST, file whitelist, strict parsers (line-numbered errors, NaN/Inf rejected), limits 256 MiB / 3M triangles / 5M rows, 300 MB body cap (HTTP 413).
- **Jobs:** one benchmark at a time with progress and **Cancel** (atomic flag, no partial results).

2. Geometry and result viewer (Task B)

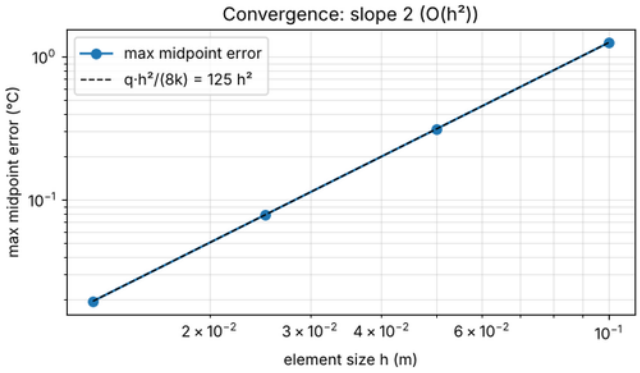


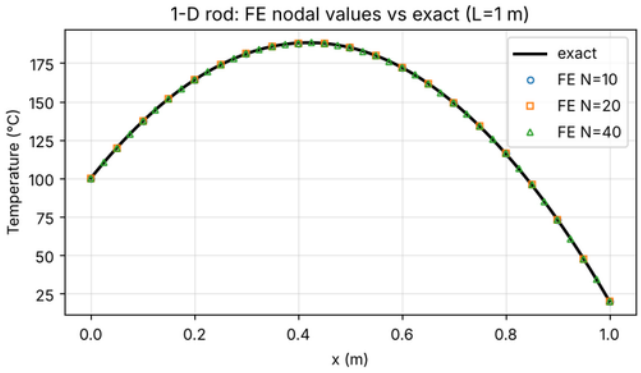
Left: ASCII STL (self-made synthetic cylinder). Right: ANSYS-style nodal CSV (`node_id,x_m,y_m,z_m,temperature_C`), colour legend, units, node inspection. The CSV is **synthetic**, not a real ANSYS export (see `data/PROVENANCE.md`). Parsing is in C++. Tested sizes: STL to 1,000,000 triangles (237 MiB, 2.0 s parse), CSV to 3,000,000 rows (122 MiB, 2.2 s); UI exercised at 100k triangles and 1M nodes; 500k triangles through HTTP in 1.6 s; larger inputs are decimated for display only (stated in the UI).

3. Thermal FEA and verification (Task C)

-kA T'' = qA on [0,L]; L=1 m, A=1e-4 m^2, k=10 W/mK, q=1e4 W/m^3, T(0)=100 C, T(L)=20 C. Two-node elements, tridiagonal assembly, Dirichlet values moved to the RHS, Thomas solve. Exact: T = T0 + (TL-T0)x/L + qx(L-x)/(2k). Midpoint error uses the **FE interpolant** $(T_i+T_{i+1})/2$ versus exact.

N	h (m)	max nodal err (C)	max midpoint err (C)	order
10	0.1	2.84217e-14	1.25	-
20	0.05	1.98952e-13	0.3125	2
40	0.025	8.81073e-13	0.078125	2
80	0.0125	1.22213e-12	0.01953125	2



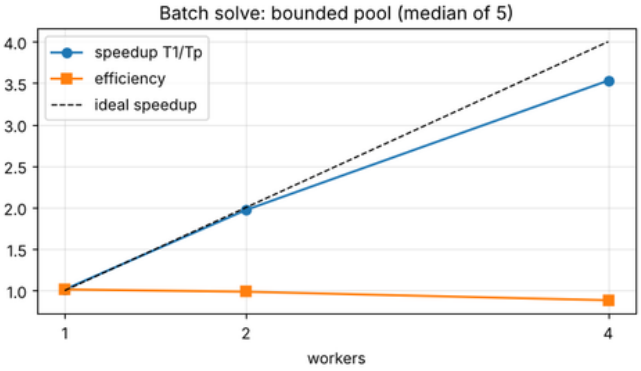


Midpoint error = q h²/(8k) = 125 h² exactly: slope 2. **Debugging insight:** nodal values are exact to round-off (1e-13), so a nodal-error check cannot reveal discretisation error or a wrong interpolation; verification therefore uses midpoint error, the convergence slope and an independent dense Gaussian-elimination solver. Three injected solver bugs (source term, two Dirichlet terms) were all caught by the tests (`tools/mutation_check.sh`).

4. Performance (Task D)

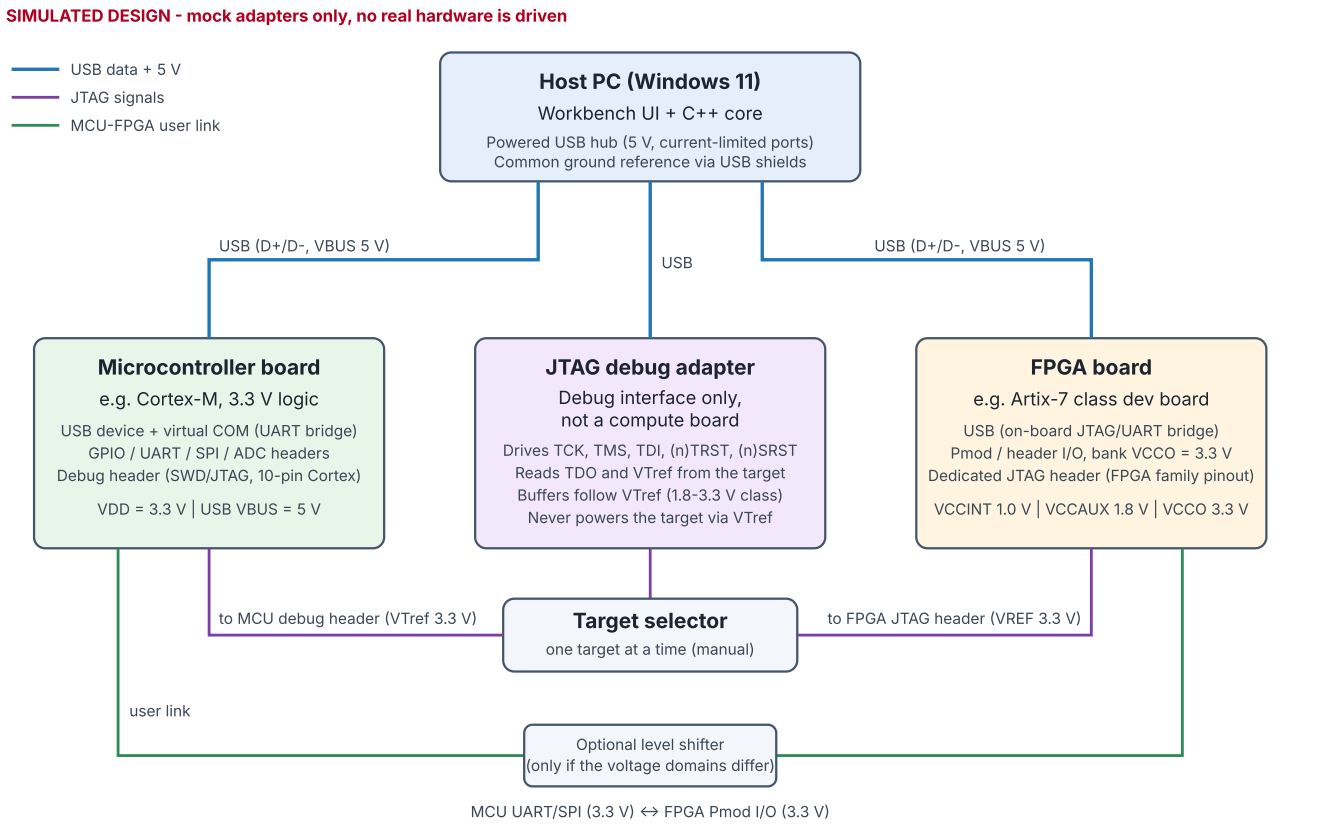
20000 cases x 500 elements, fixed seed 20260101, 1 warm-up + 5 repetitions, median; serial T1 baseline; pool of 1/2/4 workers (never above logical CPUs); pool threads created outside the timed region. Machine: 11th Gen Intel(R) Core(TM) i5-11320H @ 3.20GHz, 4 physical / 8 logical cores, 15.79 GiB RAM, Windows, GCC 13-posix, Release, flags `-std=c++17 -O3 -DNDEBUG`. Launch-to-ready: 40 ms (reported separately).

impl.	workers	median (s)	speedup T1/Tp	efficiency	max diff (C)
serial	1	0.1348	1.00	1.00	0
pool	1	0.1331	1.01	1.01	0
pool	2	0.0684	1.97	0.99	0
pool	4	0.0382	3.53	0.88	0



Serial and parallel results agree to 0 (limit 1e-8). Scaling is near-linear to the 4 physical cores (3.53x); efficiency 0.88 at 4 workers (memory bandwidth, scheduling, join). Efficiency 1.01 for 1 worker is timing noise (serial spread about 8 %). Workers beyond the logical CPU count are not offered.

5. Embedded design (Task E) - SIMULATED



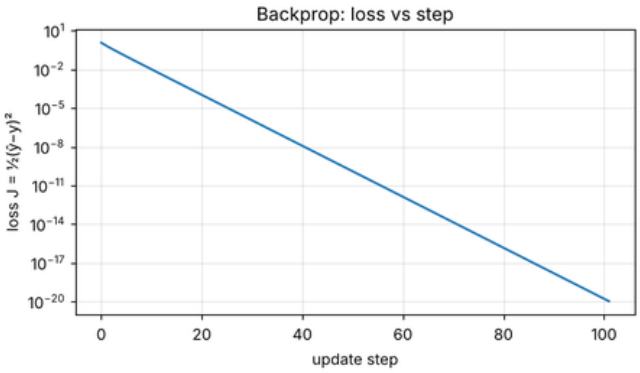
All device behaviour is simulated (C++ mocks: discovery, capabilities, connect/disconnect, one telemetry message, injected recoverable `TIMEOUT`, recovery by reconnect; screenshots 08/09). Example boards are assumptions.

	MCU board	FPGA board	JTAG adapter
Link	USB virtual COM	USB (on-board JTAG/UART)	USB; JTAG/SWD to target
Power / domains	USB 5 V -> 3.3 V VDD	VCCINT 1.0, VCCAUX 1.8, VCCO 3.3 V	USB 5 V; logic follows target VTref; never powers target
Connectors	Arduino/morpho; 10-pin Cortex debug	Pmod; 14-pin FPGA JTAG	target-side header differs per target
JTAG ownership	target: TDO, VTref, nSRST	target: TDO, VREF	adapter drives TCK/TMS/TDI/TRST/SRST, reads TDO/VTref
Ground / shifting	common GND via USB + debug GND pins	same	GND first; shifter only for 1.8 V/5 V targets

A JTAG adapter is a **debug interface, not a compute board**: it runs no application, adapts to the target's VTref, and its connector, protocol (SWD/JTAG) and pin ownership differ per target. The software discovers by type and capability and assumes no shared connector, voltage or protocol. Pin numbers are left to each board's datasheet. **Safe plan:** datasheet review, bench supply with current limit, one device at a time, ground first, read-only (IDCODE, loopback) before any write, level shifter/series resistors where domains differ, timeouts and safe disconnect. **Limits:** no real drivers, wiring, firmware or FPGA programming.

6. Backpropagation (Task F)

x=1, y=2, w=.5, b=0, v=1, c=0, lr=.1. z=0.5, h=tanh z=0.4621, ŷ=0.4621, e=-1.5379, J=1.1825. Gradients: dJ/dc=-1.5379, dJ/dv=-0.7107, dJ/dz=e v(1-h²)=-1.2095 = dJ/dw = dJ/db. Update: w=0.6209, b=0.1209, v=1.0711, c=0.1538, J=0.6858. C++ central differences (ε=1e-6): max error 1.1e-10 ≤ 1e-5 PASS. Loss falls smoothly (1.18 → 1e-2 at step 10 → 1e-20 at step 100): one sample, four parameters, so it fits exactly; this is memorisation, not generalisation. Handwritten derivation: appendix (separate pages).



7. Test outcomes

70 C++ unit tests, 0 failed (FEA 12, parsers 24, batch 12, hardware 10, backprop 8, JSON 4); 43 UI end-to-end checks passed (headless Chromium); mutation check 3/3 caught; independent cross-checks (dense solver, Python golden values for the generator, finite differences). Manual on a real Windows 11 PC (i5-11320H): `Launch.exe` opens the window without a console; benchmark results above come from that PC.

8. Limitations and AI-use disclosure

- Unsigned `Launch.exe` (SmartScreen: More info → Run anyway); hardware is simulated; 1-D thermal model only; ASCII STL only (metres assumed); software 3-D renderer; MSVC build and other Windows versions untested; limits as above.
- Claude Sonnet 5.5 (Claude Code) wrote most of the code and docs under my direction; defects it produced (infinite tick loop, single-recv health check, overflow risk, legend overlap, empty flags line, missing output folder) were found and fixed - see `docs/AI_USE_LOG.md`. The handwritten derivation is my own work. Ownership: `docs/OWNERSHIP.md`; third-party: cpp-httplib 0.18.3 (MIT).